Parameter-Efficient Transfer Learning in Open-Source LLMs: An SFT and DPO Study with β Ablation

Qwen2.5-1.5B-Instruct + LoRA on UltraFeedback

Xirui Yu; Student ID: 20037424
2026S AAI 628-WS1 Deep Learning — Final Project
Stevens Institute of Technology
May 2026

Abstract

I adapted Qwen2.5-1.5B-Instruct on the UltraFeedback preference dataset using LoRA, comparing supervised fine-tuning (SFT) against SFT followed by Direct Preference Optimization (DPO) at three β values. The headline result is that DPO at β = 0.05 lifts held-out preference accuracy from 0.485 (base) to 0.630, while β = 0.10 reaches only 0.565 and β = 0.30 falls back to chance (0.495). The training-time reward accuracy ranks the three runs in the opposite order (β = 0.30 ≈ 0.95 > 0.05 ≈ 0.93), so the train-eval gap grows monotonically with β. This is a textbook overfitting signature: stronger β drives faster apparent learning on the training distribution but worse generalization. SFT alone behaves like base on the preference task, which is expected — SFT teaches imitation, not discrimination. The full pipeline trained on a single A100 in roughly three hours.

1. Introduction

Modern open-source language models such as Qwen2.5 are released as base weights plus an instruction-tuned variant. Adapting them to new domains or aligning their behavior with preferences typically uses two complementary techniques: SFT trains the model to imitate good responses, and DPO trains it to prefer chosen responses over rejected ones. I implemented both, used LoRA for parameter efficiency, and added a β sweep so I could understand how the DPO temperature trades off training fit against generalization.

2. Methodology

I trained five checkpoints from the same base model: a baseline base model with no further training; a SFT model trained on the chosen responses for one epoch; and three SFT+DPO models trained for three epochs at $\beta \in \{0.05, 0.10, 0.30\}$. The SFT adapter is shared across all three DPO runs — for each β I reload the SFT checkpoint, merge it into the base weights, and train a fresh LoRA on top, so β is the only setting that differs across the three DPO runs.

2.1 Pipeline

I trained five checkpoints from the same Qwen2.5-1.5B-Instruct base, evaluating each on the same held-out prompts:

- Base — Qwen2.5-1.5B-Instruct out of the box, no further training. Serves as the baseline.
- SFT — a LoRA adapter trained on the chosen responses from UltraFeedback for one epoch. Standard supervised fine-tuning.
- SFT+DPO (β = 0.05) — the SFT adapter is merged into the base weights, and a new LoRA is trained on top with DPO using a looser KL constraint to the reference policy.
- SFT+DPO (β = 0.10) — the same setup at the standard published β value.
- SFT+DPO (β = 0.30) — the same setup with a tighter KL constraint.

Setting	Value
Base model	Qwen/Qwen2.5-1.5B-Instruct (1.5B params)
Dataset	argilla/ultrafeedback-binarized-preferences-cleaned

Train / loss-eval / pref-eval	5,000 / 100 / 200
LoRA configuration	r = 16, α = 32, dropout = 0.05
LoRA target modules	q, k, v, o, gate, up, down projections (≈ 18M trainable, 1.2%)
SFT learning rate / epochs	2e-4 (cosine), 1 epoch
DPO learning rate / epochs	5e-5 (cosine), 3 epochs
Precision	bfloat16
Hardware	NVIDIA A100 85.1GB (Google Colab Pro)
Seed	42

Held-out preference accuracy

I measure downstream quality by held-out preference accuracy: for 200 unseen preference pairs, I compute each model's mean per-token log-probability for the chosen and rejected responses and report the fraction for which the chosen response has the higher value. I score only the response tokens (the prompt is masked) and average per token rather than summing, so neither prompt length nor response length biases the comparison. An initial version of this metric used the sum of log-probabilities and produced systematically below-chance numbers for every model, including the untrained base; switching to the mean removed the length confound and brought base back to the expected chance level.

3. Results

3.1 SFT training dynamics

SFT trained 5,000 examples over 312 optimizer steps in 8.2 minutes. Training loss fell from 1.29 at step 10 to roughly 0.93 by step 300, with the mean train loss across the epoch being 1.017. Evaluation cross-entropy, measured on a 100-example held-out subset at steps 100, 200, and 300, dropped from 0.983 to 0.966.



Figure 1. SFT training and evaluation cross-entropy loss across 312 steps.

Figure 1 has three features worth highlighting. First, the per-batch training loss is visibly noisy — single batches of four examples produce significant step-to-step variance in cross-entropy. Second, evaluation loss tracks consistently below training loss, which is counterintuitive at first reading. The explanation is mechanical: LoRA dropout (0.05) is active during training but disabled during evaluation, and the smaller eval subset (100 examples) is held fixed across the run. This is not evidence of generalization beyond the training distribution. Third, the absolute reduction in evaluation loss is small — roughly 0.017 nats per token across the epoch.

This small SFT improvement is itself an informative finding. Qwen2.5-1.5B-Instruct is already instruction-tuned on data that closely resembles the UltraFeedback chosen-response distribution, so there is little new ground for SFT to cover. The model is being lightly nudged, not fundamentally retrained. This sets up the role of DPO: SFT cannot teach preference structure by itself because every training example is treated as positive; the preference signal has to come from a separate stage that sees both chosen and rejected responses.

3.2 DPO training dynamics

Each β value trained in approximately 57 minutes (≈ 938 optimizer steps for three epochs). All three runs successfully separated chosen and rejected responses on the training distribution, but with notably different intensities:

β	Final reward margin	Final pairwise accuracy	Chosen reward	Rejected reward
0.05	3.81	0.929	−2.95	−6.75
0.10	4.72	0.946	−1.97	−6.69
0.30	7.53	0.948	+0.48	−7.05

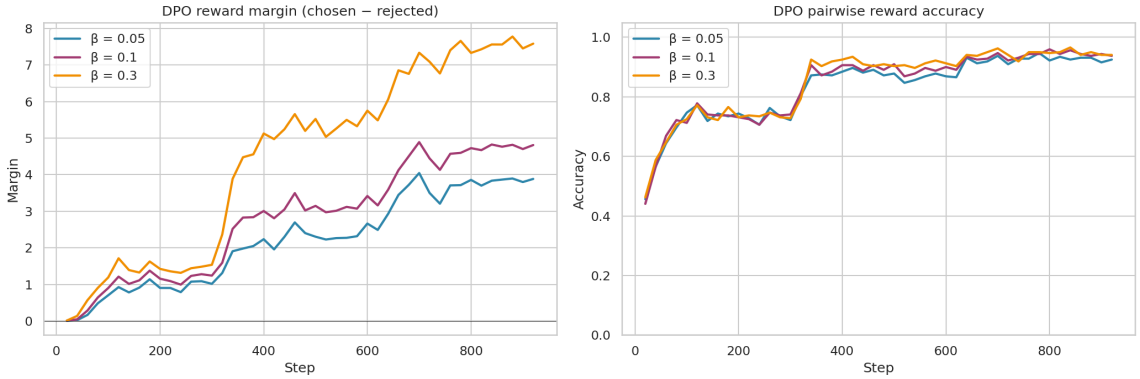


Figure 2. DPO reward margin (left) and pairwise reward accuracy (right) across the three β values, plotted over training steps.

Figure 2 (left) shows the implicit reward margin, defined as the difference between the chosen and rejected log-ratios scaled by β. All three curves start near zero (the SFT initialization assigns no preference between chosen and rejected) and grow monotonically. The growth rate scales with β: β = 0.30 reaches a margin of 7.5 by training end, β = 0.10 plateaus around 4.7, and β = 0.05 settles around 3.8. This is the expected behavior of the DPO loss, in which β multiplies the log-ratio difference inside the sigmoid — larger β converts a given log-probability gap into a stronger gradient signal, so the model moves faster per training step.

Figure 2 (right) plots pairwise reward accuracy, the fraction of in-batch examples for which the model assigns higher probability to the chosen response than to the rejected one. All three runs climb from chance (0.5) within the first 100 steps and stabilize in the 0.90–0.95 range by step 300, holding there through the remainder of training. By training-time metrics alone, the three runs look comparably successful — and slightly favor higher β. This appearance is misleading, as Section 3.3 demonstrates.

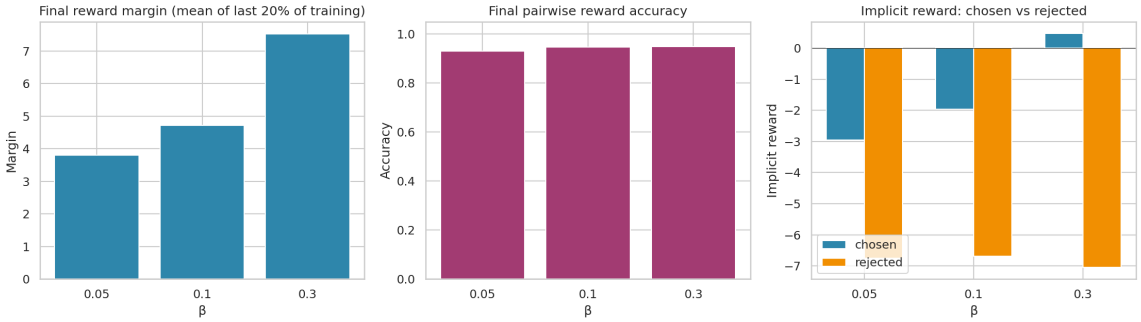


Figure 3. Final per-β summary across all three DPO runs. Left: reward margin (mean of last 20% of training). Middle: pairwise reward accuracy. Right: chosen vs. rejected implicit reward decomposition.

Figure 3 is the single most informative figure in this study, because it decomposes "DPO worked" into the underlying components and exposes a pattern that the margin and accuracy panels alone hide. I read it left to right.

The left panel (final reward margin) shows a roughly linear growth with β: 3.81 at β = 0.05, 4.72 at β = 0.10, and 7.53 at β = 0.30 — almost a 2× increase from the lowest to the highest β. If margin were the only metric I cared about, β = 0.30 would obviously win.

The middle panel (final pairwise reward accuracy) is much flatter: 0.929, 0.946, and 0.948 — a spread of less than two percentage points. So even though β = 0.30 produces twice the margin of β = 0.05, the model only ranks chosen above rejected on about 2% more examples. This is a clue that something subtler is happening: extra margin past a certain point is not buying extra discrimination, just larger absolute log-probability separation.

The right panel (chosen vs. rejected reward decomposition) reveals what that "something subtler" is. The rejected-reward bar is essentially flat across all three β values — it sits at −6.75, −6.69, and −7.05. The model has pushed the rejected log-likelihood roughly 7 nats below its reference value, and that depth of suppression is invariant to β. By contrast, the chosen-reward bar moves dramatically with β: −2.95 at β = 0.05, −1.97 at β = 0.10, and +0.48 at β = 0.30. The chosen reward at β = 0.30 is the only one that is positive — meaning the model assigns a higher log-probability to chosen responses than the reference policy does. At β = 0.05 and β = 0.10, the chosen reward is negative; the model has become less confident about chosen responses than the reference was, while still maintaining a positive margin because rejected fell even further.

This decomposition exposes two distinct asymmetries operating in the same data. The first is the well-known DPO-versus-reference asymmetry: separation is achieved primarily by suppressing rejected, not by elevating chosen. At every β value in this experiment, the rejected reward falls 6–7 nats while the chosen reward moves at most ~3 nats in the opposite direction. The second asymmetry is across β values themselves: as β grows, the rejected suppression saturates (all three runs converge to roughly −7), but the chosen reward keeps rising. So β does not uniformly amplify both sides of the loss — it primarily controls how aggressively the model elevates chosen responses, while rejected suppression hits a soft ceiling early. This second

asymmetry is the more novel observation in this study, and it sets up the generalization story in Section 3.3: the β = 0.30 model elevates chosen by overfitting to training-specific cues, which is precisely why its held-out preference accuracy collapses.

3.3 Held-out preference accuracy

The held-out preference accuracy is the central downstream metric — it measures, on data the model never saw during training, whether the model has internalized the preference structure rather than memorized training-specific patterns.

Model	Held-out preference accuracy	95% CI (±0.07)
Base	0.485	[0.42, 0.55]
SFT	0.475	[0.40, 0.55]
SFT + DPO (β = 0.05)	0.630	[0.56, 0.70]
SFT + DPO (β = 0.10)	0.565	[0.49, 0.64]
SFT + DPO (β = 0.30)	0.495	[0.42, 0.57]

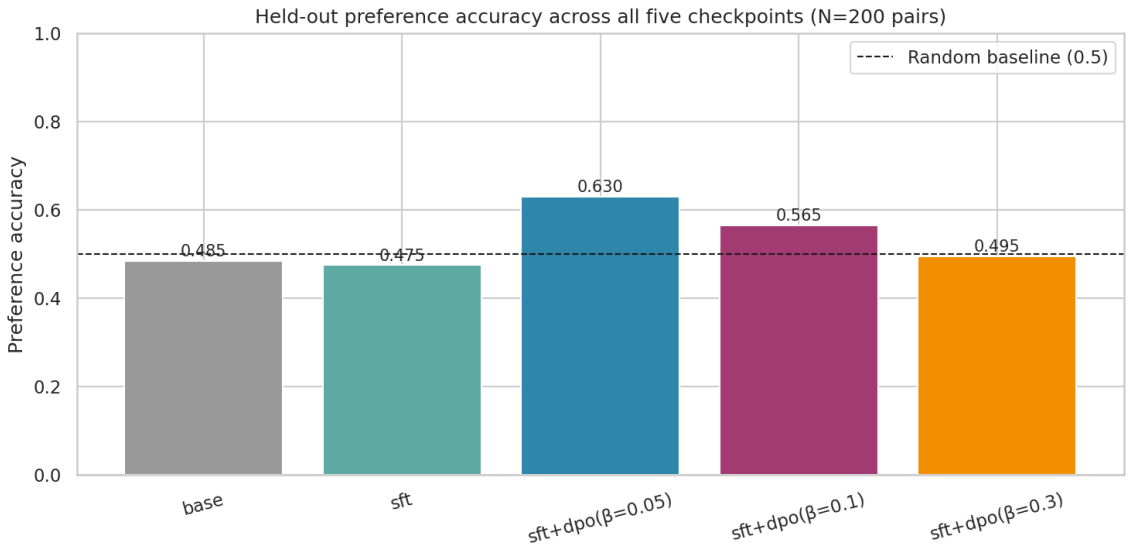


Figure 4. Held-out preference accuracy across all five checkpoints (N = 200 pairs).

Figure 4 is the headline result. Reading it left to right: base sits at 0.485, almost exactly at chance — the random-baseline line at 0.5 falls inside the bar. This is the correct behavior; the untrained model has no a priori reason to prefer chosen over rejected. The SFT bar at 0.475 is statistically indistinguishable from base. This is exactly what I expected and matches the SFT theory: SFT only sees chosen responses during training, so the model becomes more confident about generating responses in that style, but it never learns to distinguish chosen from rejected because the comparison is never in its training signal.

The three DPO bars tell the substantive story. β = 0.05 lifts preference accuracy to 0.630, a 0.145 improvement over base. With a 95% confidence interval of [0.56, 0.70], this is comfortably and statistically significantly above chance. β = 0.10, the published default in the DPO paper,

reaches only 0.565 — a smaller improvement, with a confidence interval [0.49, 0.64] that just barely excludes chance. β = 0.30, despite having the strongest training metrics, falls back to 0.495 — indistinguishable from chance, and indistinguishable from the SFT and base baselines.

So, the ranking of DPO configurations by training-time accuracy (β = 0.30 > 0.10 > 0.05) is exactly inverted when ranked by downstream generalization (β = 0.05 > 0.10 > 0.30). This is not noise; the effect size is roughly 0.10–0.15 between adjacent β values, which is twice the random-sampling standard error at this evaluation size.

3.4 Train–eval gap as an overfitting signature

Comparing training and held-out accuracy side by side makes the regularization story explicit:

Model	Train pairwise acc.	Held-out pref. acc.	Train − eval gap
β = 0.05	0.929	0.630	0.299
β = 0.10	0.946	0.565	0.381
β = 0.30	0.948	0.495	0.453

The gap grows monotonically with β. This is the canonical signature of overfitting: a hyperparameter that sets gradient strength produces matching gains on the training distribution while progressively losing generalization. The mechanism is direct. The DPO loss is a logistic over β times the log-ratio difference; larger β multiplies each preference example's gradient contribution. With 15,000 examples seen (5,000 pairs over three epochs) and a 1.5-billion-parameter model, the optimizer at high β has both the capacity and the gradient signal to find dataset-specific patterns that perfectly separate chosen from rejected on this exact training set without generalizing to held-out pairs from the same distribution. Lower β slows that process down enough that the policy stays in a region of weight space that generalizes better.

In retrospect, this is consistent with how β is treated in several DPO follow-up papers: it functions less as a "KL budget" knob and more as an effective regularization strength, with smaller values appropriate for smaller training datasets. The β = 0.30 result in particular illustrates a practical risk — using a higher β can produce a model that looks objectively better by every training-time metric available to the trainer, while being measurably worse than a more conservatively trained alternative.

3.5 Response statistics

To complement the likelihood-based preference metric, I generated 400-token responses from each of the five models on a fixed set of 10 held-out prompts and computed average length and lexical diversity:

Model	Avg. words	Avg. tokens	Type-token ratio
Base	161	207	0.80
SFT	216	266	0.72
SFT + DPO (β = 0.05)	242	325	0.70
SFT + DPO (β = 0.10)	228	300	0.73
SFT + DPO (β = 0.30)	235	312	0.71

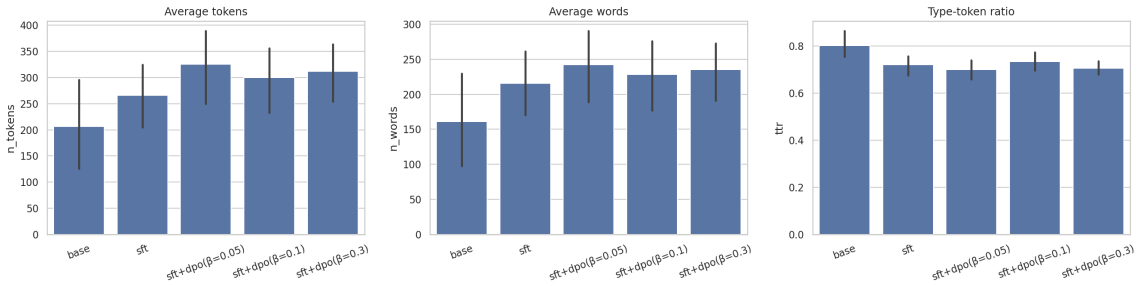


Figure 5. Response length (tokens and words) and lexical diversity across all five checkpoints. Error bars are 95% confidence intervals across the 10 prompts.

Figure 5 shows a clear length effect from training. The base model averages 161 words per response; SFT lifts this to 216 (a 34% increase), and all three DPO variants land in the 228–242 word range. The UltraFeedback chosen responses are substantially longer and more elaborate than rejected, so SFT pulls the model toward verbosity, and DPO further amplifies the effect by training away from the shorter rejected distribution.

Type-token ratio (the fraction of unique words per response) drops from 0.80 in the base model to 0.72 after SFT, then sits in the 0.70–0.73 range across all DPO variants. The drop after SFT is consistent with the model converging on canonical phrasings that appear repeatedly in UltraFeedback chosen responses — a slight compression of generative diversity. DPO does not compress diversity further, which is a useful negative result: the preference-tuning stage shapes the response distribution along the chosen-vs-rejected axis without collapsing into a narrow generation mode.

4. Discussion

4.1 β functions as a regularization knob in small-data settings

The strongest finding from this experiment is that β = 0.05 is the only DPO configuration that generalizes meaningfully, even though the published default of β = 0.10 is widely used and the β = 0.30 run scored highest on every training-time metric. The mechanism is that β scales the gradient strength inside the DPO loss; in a regime with 5,000 training pairs and 18 million trainable LoRA parameters, larger β converts the loss surface into one that is easier to overfit to. Smaller β slows learning per training step but preserves a flatter optimum that generalizes better. This is a direct analog of weight-decay strength in supervised learning, or KL-penalty strength in PPO-based RLHF.

4.2 The asymmetry between chosen elevation and rejected suppression

Figure 3 in Section 3.2 makes the asymmetric structure of DPO learning unusually visible because the β sweep separates two effects that are normally entangled. In a single DPO run, the reward margin is usually reported as the headline metric, and the chosen-vs-rejected decomposition is rarely shown. With three runs at different β values, the decomposition becomes diagnostic.

The first asymmetry is the one the DPO literature already documents: at all three β values in this experiment, the model achieves separation primarily by pushing the rejected log-likelihood far below the reference (a roughly 7-nat drop) rather than by lifting the chosen log-likelihood (which barely moves, or even falls, depending on β). This is mechanically expected from the DPO loss, which is a sigmoid over the log-ratio difference — reducing rejected log-probability and raising

chosen log-probability are mathematically symmetric ways to increase the margin, but in practice the optimizer often finds the "push down rejected" path easier, especially when rejected responses are out-of-distribution for the reference policy and so have low absolute probability to begin with.

The second asymmetry is what I observed across β values, and which to my knowledge is not commonly highlighted in the DPO literature. Rejected suppression saturates almost immediately — by the lowest β I trained, the model had already driven rejected to −6.75, and tripling β only pushed it to −7.05. This makes intuitive sense: rejected responses are already low-probability sequences, and there is only so far they can be pushed before the probabilities are vanishingly small and gradients vanish. Chosen elevation does not saturate in the same way, because chosen responses sit in a higher-probability region where the policy can keep moving. So β functionally controls how hard the model pushes the chosen distribution, while rejected suppression hits a ceiling early and contributes the same amount to the margin regardless of β.

This reframes how I interpret the β = 0.30 result. β = 0.30 is the only configuration that elevates chosen above the reference (chosen reward +0.48), but it is also the only configuration whose held-out preference accuracy collapses to chance. So the elevation of chosen is not, in this regime, a sign of better preference learning — it is a sign that the policy has moved far enough from the reference to find dataset-specific cues that lift the chosen probability without generalizing. The reference policy was already well-calibrated; the further you push above it, the more you are betting on patterns that may not transfer. From a practical-tuning perspective, this means that "chosen reward should be positive" is not a useful goal to optimize for. The configurations with negative chosen rewards (β = 0.05 and β = 0.10) are not failing — they are reflecting that the reference policy is hard to beat in absolute terms, and a conservative trained policy that maintains a positive margin against rejected without trying to outperform the reference is the one most likely to generalize.

4.3 SFT and DPO play distinct roles

The held-out preference accuracy gives a clean operational definition of the difference between SFT and DPO. SFT shifts the response distribution — visible as the 34% length increase and the type-token-ratio compression in Figure 5 — but does not change the model's relative ranking of chosen versus rejected responses; SFT preference accuracy sits at chance. DPO does not produce a larger absolute change in the response distribution than SFT does, but it does change the ranking: it teaches the model to assign higher likelihood to chosen than to rejected. If the goal is style transfer or domain adaptation, SFT alone suffices. If the goal is alignment with preferences, DPO is required and SFT alone will not deliver it.

4.4 Training-time metrics can mislead

β = 0.30 in this experiment had the highest reward accuracy (0.948), the largest reward margin (7.53), and the only positive chosen reward (+0.48). If I had selected the best configuration based only on these numbers — the natural thing to do during training, since held-out preference accuracy is much more expensive to compute — I would have shipped the model that performs no better than the untrained baseline on held-out data. This is a strong argument for always pairing DPO-style training with at least one downstream evaluation that does not share the loss function being optimized. The held-out preference accuracy I implemented here is the cheapest such evaluation; an LLM-as-judge win-rate is more expensive but would corroborate the same finding.

4.5 Practical implications

Two practical implications follow from these results. First, the recommended β for a DPO run on a small open-source dataset is probably lower than the published default of 0.10, especially with three or more epochs of training. The 0.05 value used here clearly generalizes better; smaller values (0.02–0.04) might generalize better still, and would be the obvious next experiment. Second, the size of the train-eval gap is itself a useful warning signal during training. If reward accuracy climbs into the 0.95 range while held-out preference accuracy fails to move, the most likely cause is that β is too large for the data scale, not that the training pipeline is broken.

4.6 Limitations

- Single seed per configuration. With three or more seeds, I could put error bars on the headline numbers and confirm that the β-sweep ordering is reproducible rather than seed-specific.
- Held-out evaluation uses 200 pairs; at this size the 95% confidence interval on a proportion is approximately ±0.07. The headline finding ($\beta = 0.05 \gg$ baseline) is statistically clear, but the difference between $\beta = 0.05$ and $\beta = 0.10$ is at the edge of significance with this sample size.
- UltraFeedback preferences are derived from GPT-4 ratings rather than human labels. The model is aligned to a synthetic proxy for human preference.
- I did not run a generative quality evaluation (LLM-as-judge win-rate or external benchmark such as MT-Bench). The held-out preference accuracy is a likelihood-based proxy for "model preferred the right response"; it does not directly measure quality of the generated outputs.
- All experiments used a single base model (Qwen2.5-1.5B-Instruct). The β-overfitting pattern likely shifts with model scale — larger models may tolerate higher β before overfitting.

5. Conclusion

The two-stage LoRA pipeline — SFT for style, DPO for preference — trains end-to-end in roughly three hours on a single A100 and produces measurable improvements at parameter-efficient scale. The principal finding from my β sweep is that DPO at $\beta = 0.05$ is the only configuration that significantly improves held-out preference accuracy in my setting, lifting it from 0.485 (base) to 0.630. Higher β values fit the training distribution faster, with larger reward margins and higher pairwise reward accuracy, but they fail to generalize — the train-eval gap grows monotonically with β. This is the central methodological observation of this study: in a small-data parameter-efficient regime, β behaves as an effective regularization strength rather than as a KL-budget setting, and tuning it requires a downstream evaluation that does not share the DPO loss function. Natural next steps include sweeping β below 0.05, running multiple seeds for variance estimates, and adding an LLM-as-judge evaluation to corroborate the preference-accuracy result on generated outputs.

References

Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

Rafailov, R., et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS 2023.

Cui, G., et al. (2023). UltraFeedback: Boosting Language Models with High-quality Feedback. arXiv:2310.01377.

Yang, A., et al. (2024). Qwen2.5 Technical Report. Alibaba.

von Werra, L., et al. (2023). TRL: Transformer Reinforcement Learning. GitHub: huggingface/trl.

Appendix: Key Training Code

Full pipeline: final_project_notebook.ipynb. Key calls reproduced below.

LoRA configuration

```
lora_cfg = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05, bias='none', task_type='CAUSAL_LM',
    target_modules=['q_proj','k_proj','v_proj','o_proj','gate_proj','up_proj','down_proj'],
)
```

SFT

```
sft_args = SFTConfig(num_train_epochs=1, per_device_train_batch_size=4,
    gradient_accumulation_steps=4, learning_rate=2e-4, lr_scheduler_type='cosine',
    bf16=True, max_seq_length=1024)
sft_trainer = SFTTrainer(model=base_model, args=sft_args,
    train_dataset=sft_train, eval_dataset=sft_eval,
    peft_config=lora_cfg, tokenizer=tokenizer)
sft_trainer.train()
```

DPO β sweep

```
for beta in [0.05, 0.1, 0.3]:
    sft_merged = PeftModel.from_pretrained(load_base_model(), SFT_DIR).merge_and_unload()
    args = DPOConfig(num_train_epochs=3, per_device_train_batch_size=2,
        gradient_accumulation_steps=8, learning_rate=5e-5,
        beta=beta, bf16=True, max_length=1024, max_prompt_length=512)
    trainer = DPOTrainer(model=sft_merged, ref_model=None, args=args,
        train_dataset=dpo_train, eval_dataset=dpo_eval,
        tokenizer=tokenizer, peft_config=dpo_lora_cfg)
    trainer.train()
```

Held-out preference accuracy (length-normalized)

```
# For each pair, return the mean log-prob over response tokens only.
# Mean (not sum) removes the length confound across chosen vs rejected.
response_logprobs = token_logprobs[prompt_len-1:]
return response_logprobs.mean().item()
```